THINKST SCAPES

Q2 • 2025

https://thinkst.com/ts

Contents

Cover photo: *Black Canyon of the Gunnison National Park, Colorado. Image by Jacob Torrey (Thinkst)*

Introduction

Welcome to the Quarter 2, 2025 edition of ThinkstScapes! This issue focuses on content released, published or presented between the first of April and the end of June 2025.

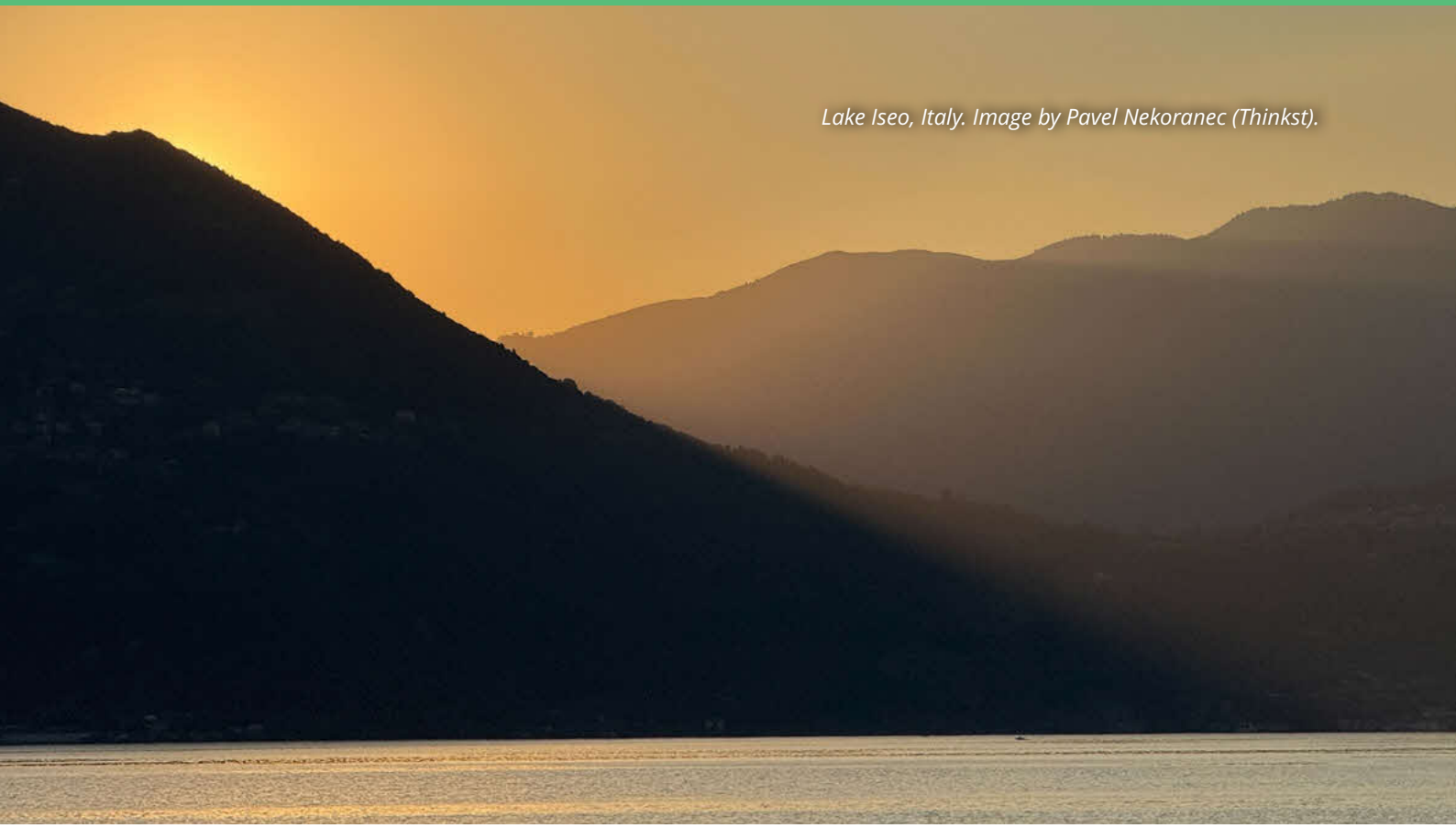
Q2 showed an uptick in the quantity of published content over Q1. Historically, Q2 sees less research, with the lion's share saved up for the Hacker Summer Camp trio of conferences in Q3. This year however, we're seeing European conferences picking up that slack, perhaps as traveling to the US is perceived as riskier. Regardless of venue, there's some great material to cover this quarter!

As a reminder: if you are aware of work we've missed, a blog post we should have seen or a conference we should have covered, we'd love to hear about it. Please send them to **ts@thinkst.com**! We'd like to thank Casey Smith for his tip on Local Mess, featured in this issue.

Uroa Beach, Zanzibar. Image by Paul Gichuki (Thinkst).

In addition to almost 1,400 blog posts, this quarter's content was drawn from talks and papers presented at the following conferences:

Conference name	Number of talks/papers
BSidesKC	14
BSides Exeter	28
BSides Adelaide	14
BSidesCharm	26
BSides Luxembourg	22
ELBSides	13
BSides SATX	27
CONFidence	44
Black Hat Asia	48
BSidesSF	103
Zer0Con	11
Offensive Con	17
RMISC	67
BlueHat IL	32
RVASec	33
ShowMeCon	23
x33fcon	24
TROOPERS	34
fwd:cloudsec	43
BlueHat India	14
SecretCon	32
Cyphercon	44
IEEE S&P	255
LangSec Workshop	14
REcon	29
ACNS	55
RSAC	394
Total	**1,460**



Lake Iseo, Italy. Image by Pavel Nekoranec (Thinkst).

Themes covered in this issue

NETWORKING IS ALWAYS TRICKY

Assumptions can be dangerous, especially when they pertain to network security. This theme highlights four works, highlighting a range of issues from scanning the internet through firewalls to confusions about localhost, and how to hide from TLS censorship.

LANGUAGE MODELS LARGE AND SMALL

LLMs and their impact on security are here to stay. We feature six works showing different aspects of how models can change the security landscape, generate insecure code and find vulnerabilities and secrets in code. Lastly, to secure the models themselves, we feature work that can find backdoors in LLMs.

WHEN PARSING GOES RIGHT, AND WHEN IT GOES WRONG

Understanding data is core to information security. This theme features four works on how AI can help generate secure parsers and extract specifications from binaries, and how to take advantage of parser differentials to bypass security boundaries.

NIFTY SUNDRIES

As always there's been some great work that doesn't fit into our above themes. This quarter, you can look for: pretending to be an Apple device to leak MDM server data, using serial cables as antennas, finding bugs in memory-safe code, IDE attacks, and bring-your-own-cloud security considerations. We end with a promising standard from the telecom industry to replace SMS OTPs with a secure (and seamless) alternative.

Networking is always tricky

- *Beyond the Horizon: Uncovering Hosts and Services Behind Misconfigured Firewalls*
- *0.0.0.0 Day: Exploiting Localhost APIs From The Browser*
- *Local Mess: Covert Web-to-App Tracking via Localhost on Android*
- *Transport Layer Obscurity: Circumventing SNI Censorship on the TLS-Layer*

Antelope Canyon, Arizona, USA.
Image by Vittoria Tosa (Thinkst).

Beyond the Horizon: Uncovering Hosts and Services Behind Misconfigured Firewalls

***Authors**: Qing Deng, Juefei Pu, Zhaowei Tan, Zhiyun Qian, and Srikanth V. Krishnamurthy*

Paper

This research systematically scanned the IPv4 internet space for firewall misconfigurations in order to understand their prevalence and to examine the vulnerabilities of the inadvertently-exposed services. While most modern firewalls operate via stateful rules to only allow external traffic through the firewall when associated with an internally-initiated request, firewalls can be (or configured to be) stateless. This lack of tracking of connections (or related UDP packets) means that if a stateless firewall allows TCP connections to, e.g., port 80, it must also allow externally-generated traffic with a source port of 80 through. This led to a common misconfiguration in the past where stateless firewalls permitted source port 80 or source port 53 traffic to be allowed inbound from any external IP when permitting explicit outbound access. This vulnerability was publicised widely in 2014 as both Microsoft and Apple's firewalls were susceptible; the popular nmap scanner even has a feature to specify a source port for scans.

This research worked to understand the prevalence of these misconfigurations a decade later, by scanning the entire IPv4 address space using commonly used source ports. For each detected service, it was then rescanned multiple times with random high value ports to determine if the service was publicly accessible, or was only accessible with this vulnerability. The study found almost 2.5 million newly accessible services running on over 2 million IP addresses that had not responded to previous scans. These services were then analysed to determine if their security posture was worse than a sample of publicly-reachable services. As one would expect, the researchers found a high prevalence of old OSes, weak ciphersuites, disabled authentication, and known-vulnerable services in the firewalled group.

Lastly, the researchers tried to determine whether this vulnerability is well-known by attackers. They launched honeypots across the internet behind stateless firewalls and monitored for connections to services. The data from the honeypots and the lack of ransomware signatures found on firewalled MongoDB services indicate that this trick has been lost (for now) to the annals of history, and is not common in-the-wild.

Figure 1.
A table showing the number of non-publicly discoverable services reachable by setting the scanner's source port to either 80 (for TCP), or 53 (for UDP).

Service	Port	Count
SSH	TCP 22	234,984
Telnet	TCP 23	50,820
RDP	TCP 3389	7,931
IPMI	UDP 623	4,242
SNMPv1		42,894
SNMPv2c	UDP 161	36,753
SNMPv3		465,033
FTP	TCP 21	32,172
SMB	TCP 445	19,419
MySQL	TCP 3306	19,456
MongoDB	TCP 27017	338
HTTP	TCP 80	222,539
HTTPS	TCP 443	193,630
DNS	UDP 53	334,358
NTP	UDP 123	824,389

TAKEAWAYS:

- **While the source port manipulation technique** was known, the rigour of testing it across the entirety of IPv4 space and classifying the services discovered is valuable itself.
- **It makes sense** that patching and security investments are prioritised on public-facing systems, and that the security posture of inadvertently-exposed systems would be worse. While IPv6 was excluded from this research, research on scanning IPv6 is progressing – coupled with this technique, it could expose many more devices that are behind NATs and stateless firewalls.
- **While the research showed limited evidence** of malicious usage of this technique in the wild, that is not going to last. Check your firewall rules, patch your systems, and add authentication controls – even on internal systems!

0.0.0.0 Day: Exploiting Localhost APIs From The Browser

***Authors**: Avi Lumelsky and Gal Elbaz*

Blog *Video*

Local Mess: Covert Web-to-App Tracking via Localhost on Android

***Authors**: Aniketh Girish, Gunes Acar, Narseo Vallina-Rodriguez, Nipuna Weerasekara, and Tim Vlummens*

Website

These two works show some of the nuanced security issues relating to how browsers can both run untrusted code, and access unprotected local network assets. **The first talk** shows that while some protections have been built into browsers for remote sites accessing client local ports, differences in how the address 0.0.0.0 is understood means that, until very recently, those protections could be bypassed. The consequences of this weakness are far reaching, allowing a malicious website to perform command injection into locally-hosted tools. As a proof-of-concept, the distributed computing framework Ray was used, which deploys a daemon listening on localhost for commands without authentication. Malicious Javascript could then issue fetch commands to control the worker.

The second research release is the discovery of covert coordination between commonly-installed Android applications and website pixeling scripts to track and correlate browsing between browsers and third party cookie restrictions. The Yandex and Facebook applications would listen locally on the Android device, and their respective pixeling scripts would share unique identifiers to correlate the user. This would allow tracking even in private (incognito) browsing modes or when not logged-in via the browser. While Yandex's technical approach was simply issuing HTTP requests to the localhost ports, Facebook used the WebRTC protocol to connect to the native application. After disclosure, mobile browsers have tightened access to localhost ports, and Facebook's pixel scripts have stopped trying to connect to localhost (though without any acknowledgement of this previous behaviour).

TAKEAWAYS:

- **Both of these works** highlight the clashing security models in play with how a browser restricts remote scripts, and how localhost networked IPC allows for seamless communication among software components.
- **The confusion** around how 0.0.0.0 was meant to be used (and understood) meant that systems were vulnerable for almost two decades.
- **That the** `INTERNET` **permission** gives Android applications the ability to open listening sockets is difficult to contextualise to users and will show up again as a target of abuse.
- **Sadly, Facebook was all too keen** to monopolise on this weakness until caught. Luckily, vigilant researchers helped keep user privacy in check.

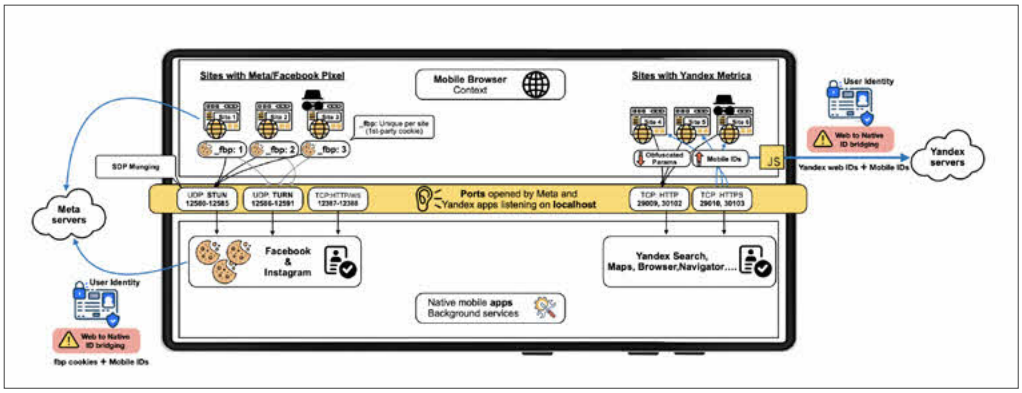


Figure 2. *A figure showing how Facebook and Yandex applications on Android could correlate web browsing traffic through local port listeners.*

Transport Layer Obscurity: Circumventing SNI Censorship on the TLS-Layer

***Authors**: Niklas Niere, Felix Lange, Juraj Somorovsky, and Robert Merget*

Code

Paper

This research explored TLS censorship systems and manipulations at the TLS level to bypass such systems while still interoperating with unmodified servers. Blocking specific IP addresses and unencrypted traffic flows is simple enough, but the growth of encrypted traffic and IP addresses that host multiple services makes censorship a tougher challenge. As of TLS 1.3, support is mandated for the SNI field, which is used by a client to specify which of potentially many websites hosted on the same server it would like to establish a connection with.

The researchers started developing a tool to detect censorship of TLS, as well as modify certain parameters of the connection. These modifications were then tested against a variety of different web servers and configurations to determine which would allow for successful HTTPS browsing. Finally, each attempted bypass modification was evaluated against both China and Iran's censorship systems (of which China has multiple) while still allowing for browsing popular sites.

The most successful approach exploits the stateless nature of such edge censorship systems. These systems see traffic as it flows by and can decide to inject a RST packet to kill the connection. By using TLS fragmentation, the SNI field can be split across multiple packets, none of which match a blocked domain, and thus are able to circumvent the block.

TAKEAWAYS:

- **Fragmentation and out-of-order reassembly** offers a powerful primitive to bypass network filters. Such filters (e.g., censorship devices, WAFs, etc.) either must operate statelessly, or with a QoS guarantee that allows for crafting of fragment sequences that can bypass many checks.
- **While as an anti-censorship technique** it is a tool for good, similar techniques can be used to bypass perimeter defenses.

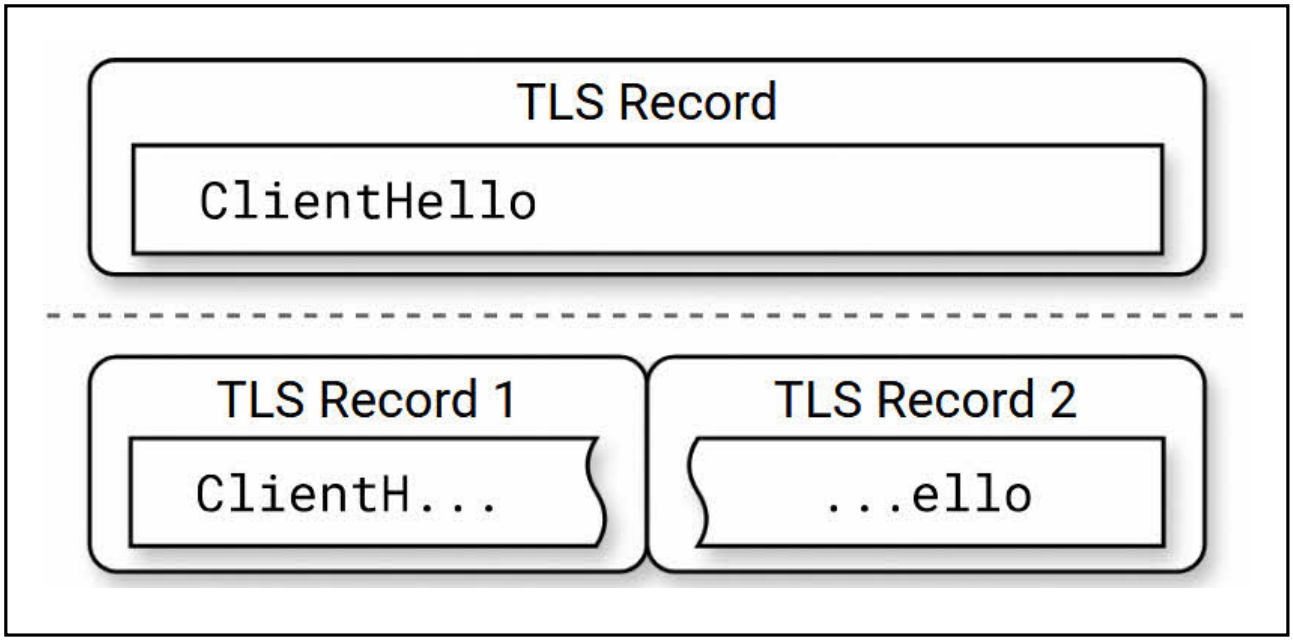


Figure 3.
A figure demonstrating the rarely-used (but protocol-compliant) feature of TLS fragmentation. By splitting the TLS header and ClientHello across multiple packets, censorship systems can be circumvented.

Language models large and small

- *The road to Top 1: How XBOW did it*
- *AI and Secure Code Generation*
- *A look at CloudFlare's AI-coded OAuth library*
- *How I used o3 to find CVE-2025-37899, a remote zeroday vulnerability in the Linux kernel's SMB implementation*
- *Enhancing Secret Detection in Cybersecurity with Small LMs*
- *BAIT: Large Language Model Backdoor Scanning by Inverting Attack Target*

Devon Valley Wines outside Stellenbosch, South Africa. Image by Roberto Aldera (Thinkst).

The road to Top 1: How XBOW did it

***Author**: Nico Waisman*

Blog

AI and Secure Code Generation

***Authors**: Dave Aitel and Dan Geer*

Blog

These higher-level blog posts explored the relationship between LLMs and software security. Starting with the staggering statistics that a quarter of new code written at Google is LLM-generated, and the number one HackerOne US reporter is an autonomous LLM agent, the authors explore the implications of LLMs improving in their ability to reason about security.

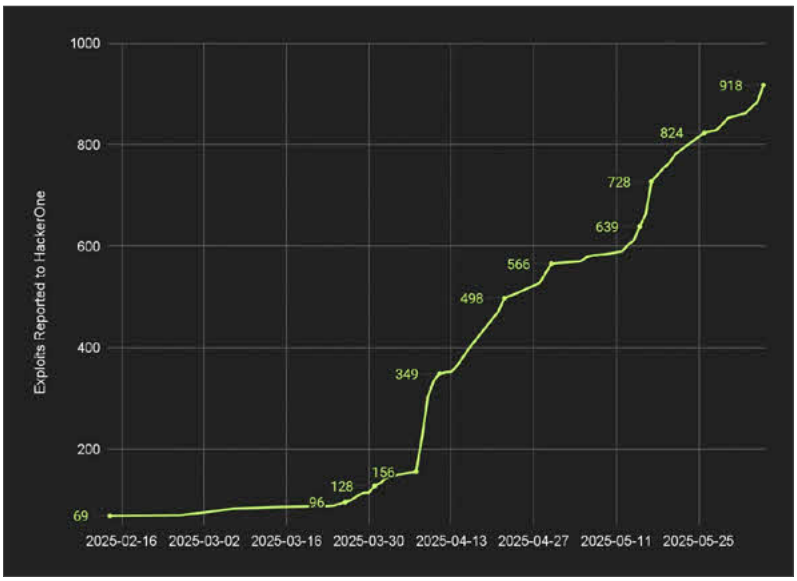
The first blog post shows that it's possible to scale up LLM vulnerability discovery and reporting while managing false positives and noise. The second post posits that LLMs must be given more autonomy in higher-level security posture improvements, if defenders want to keep pace with machine-augmented attackers. When it comes to determining the impact of a vulnerability on a specific system or organisation and autonomously mitigating weaknesses, LLMs will be given more control, and defenders will need to live with reduced visibility into decision-making processes.

Current models to determine the impact of a vulnerability resort to static measures such as software bill-of-materials and the CVSS scoring rubrics. While these allow for a rough estimate, it currently takes significant manual effort to determine if a newly-discovered vulnerability will impact a specific organisation. If, for example, the vulnerability is in a library, analysts must determine how the library is used, which functions are reachable, and if there are other mitigations in place that reduce impact. As more system artefacts are accessible to LLMs, this nuanced analysis can be offloaded (partially or perhaps in full).

TAKEAWAYS:

- **While readers looking for hard technical details** will be disappointed by these posts, they allude to the dramatic shifts coming to our industry as LLM-based automations improve.
- **As XBOW continues to scale up,** their findings could indicate if bugs are sparse or dense. If the limitations of human time are removed, will XBOW continue to find bugs in well-examined software (dense), or have to move to other projects to find new vulnerabilities (sparse)? This sparse vs. dense determination can have big consequences for policy-makers; it will be exciting to see more of the data as promised in the blog post.
- **LLM-assisted impact analyses** will only get better as context windows continue to increase in size, and more system data is codified. Human-language design documents can be ingested alongside infrastructure-as-code source and software components to perform whole-system analyses. Expect to see better impact analyses in the future and a shift away from relying on a static CVSS score.

Figure 4.
A chart showing the rapid growth of LLM-reported real security findings to HackerOne by XBOW.



A look at CloudFlare's AI-coded OAuth library

***Author**: Neil Madden*

Blog

This blog post, by an experienced OAuth security developer, examined a new OAuth library built mainly by an LLM. CloudFlare, the library creator, noted that this library was a human-LLM teaming effort, and that every line was human reviewed. The commit logs for the library also detail each modification as human or LLM-sourced.

At first glance, the library was designed with a solid foundation, but diving deeper, flaws became evident. Overall, the author noted insufficient testing code, missing security best-practices, and a few instances where OAuth-specific functionality was implemented incorrectly. Looking at the commit logs provided more context where human insights were added to the prompt, and where the humans found serious security flaws in the candidate implementation. Overall, the library suffers from the disparity of human knowledge when reviewing the code. The author suspects that since there are ample resources online (and in training corpuses) that are out-of-date or flat out wrong, the LLM code was influenced negatively.

TAKEAWAYS:

- **Software development education** focuses on the design and writing of software. Reading and building context in existing (or LLM-generated) code is not as emphasised.
- **As seen in the commit logs** during the development of the OAuth library, the developers were able to contribute sound designs in the initial prompting, but missed errors in the resultant output.
- **Without sound tools** to verify the code generated by models, even humans with the best of intentions will miss things.
- **It is worrying** to see an untested development model and laissez-faire review process applied to a core authentication library. Expect to see lots of catastrophes until development communities learn how to safely adopt these new tools.

How I used o3 to find CVE-2025-37899, a remote zeroday vulnerability in the Linux kernel's SMB implementation

***Author**: Sean Heelan*

Blog

Code

Use-after-free

In programming languages where developers manage memory directly (e.g., C, C++), use-after-free (UAF) is a bug class where a pointer to a region of memory is used after the memory has been freed and potentially reallocated. Memory corruption on the program's stack has become less exploitable in modern systems due to hardware protections, and power bug-finding tools that look for spatial memory violations. UAF bugs occur in the program's heap and are harder to find because there's a temporal violation. Because these bugs corrupt memory usage on the heap they can provide remote code execution if an attacker can control the memory allocated to the previously-freed region and the pointer used after free is a function pointer.

UAFs account for approximately 20% of all memory corruption CVEs, and are difficult to find automatically, so they can demonstrate the power of a tool or technique.

This blog post covered the evaluation of OpenAI's GPT-o3 for vulnerability discovery, and in the process of evaluation discovered a new zero-day in the Linux kernel. The author used a known use-after-free vulnerability in the kernel's SMB file-sharing implementation to measure o3's ability to find the vulnerability. The LLM was provided the bulk of the source code, a description of how it works, and a prompt to look for use-after-frees. Each experiment was repeated 100 times to estimate the success in the face of the LLM's non-determinism. With a smaller context window (i.e., less irrelevant code to the vulnerability), the performance was improved, but with more manual upfront effort to trim the code.

While evaluating o3 with more context, a real, unknown use-after-free was discovered among the false positives (where the LLM would report a vulnerability that did not exist in the code). This finding, despite being only found in one of the 100 executions, shows that there is promise in the integration of LLMs into the vulnerability research process, with automation needed to weed out the noise of false positives.

TAKEAWAYS:

- **This post highlights** that the value of LLMs is non-deterministic, and, like other seed-based tools (e.g., fuzzers), should be run repeatedly.
- **Automation to validate outputs** is essential to manage the false positives coming from LLMs, just like when dealing with over-approximations from static analysis tools.
- **More work is needed** to quantify how many inferences are needed to cover a region of code with LLMs.
- **This blog helps show** how changing context size can influence performance, but more experimentation is needed to determine the tradeoffs of inferences, context size and performance.

Enhancing Secret Detection in Cybersecurity with Small LMs

***Authors**: Danny Lazarev and Erez Harush*

Blog

Video

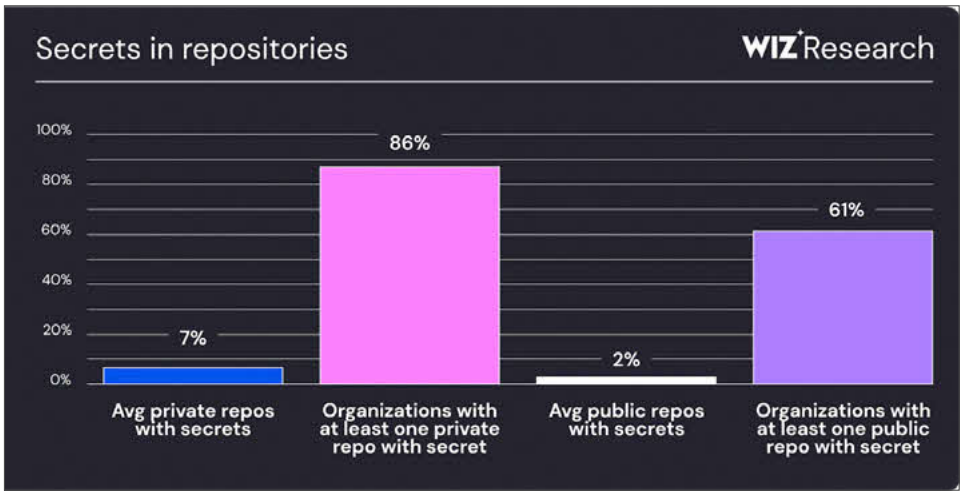
This talk detailed the process of building a secrets scanner for code with a fine-tuned language model. The authors note that in almost a third of reported breaches, stolen or leaked credentials were the initial access vector, and that a majority of organisations have at least one secret in a private repository. Currently, secrets scanning is heavily reliant on regular expressions, which work well for certain types of secrets, but can miss secrets in logs and comments, or falsely report strings that match a pattern but are not actual secrets. LLMs are better able to understand the context of source code and outperform regular expressions, but they come with scalability, cost and privacy drawbacks.

The researchers explored taking a smaller, 1B parameter version of Meta's Llama 3.2 and fine-tuning it to recognise secrets that existing scanners typically miss. By quantising this smaller model, the researchers found that it performed better than regular expression-based scanners, and could run locally on a small ARM CPU without the need for a GPU or other accelerator. Other open-weight models were evaluated, with Llama hitting the right mark of performance and compute cost.

TAKEAWAYS:

- **This model of fine-tuning** a smaller model removes most of the security concerns about deployment.
- **The privacy, scalability and cost benefits** mean that this model should be considered for a variety of use cases.
- **As more use cases of language models** are optimised into more specific smaller models, expect to see standardised pipelines or approaches gain acceptance – allowing for a flourishing ecosystem of specialised models for private edge deployment.

Figure 5. *A chart showing that secrets are found in the private repositories of most organisations.*



BAIT: Large Language Model Backdoor Scanning by Inverting Attack Target

***Authors**: Guangyu Shen, Siyuan Cheng,*

Code

Paper

This work aims to detect if an LLM has been trained or fine-tuned maliciously to add a backdoor. Backdoors in LLMs are specific input tokens (or sequences thereof) that cause the LLM to output an attacker-selected sequence of tokens. For example, an LLM could be backdoored to always `import` an attacker-generated Node.JS package, but only when prompted to generate a Node.JS source file. Existing detection algorithms for backdoored LLMs are slow and inaccurate.

BAIT can search a black-box model (i.e., closed-source), and with high-accuracy determine if there are triggering inputs that cause the model to generate specific (lower-entropy) outputs. Backdoors cause a deviation from the normal probability distribution and entropy of responses. The tool iterates through the input token set looking for specific tokens (or sequences thereof) that will cause the entropy of responses to drop significantly. Over a varied host of different LLM types and backdoor architectures, BAIT averages 98% accuracy with an average runtime of under 15 minutes.

TAKEAWAYS:

- **As the ecosystem** around customised/fine-tuned models matures, these types of tools that provide some ability to inspect models are needed.
- **CI/CD pipelines** ingest copious amounts of third-party code, but at least there are tools and the ability to inspect that code. Models are opaque blobs: without robust tooling to inspect and validate their behaviour, it will be impossible to completely trust any third-party model.
- **The scale of investments needed** to train modern LLMs prohibit validation/recreation of any performant model – there is no feasible way to create a process analogous to reproducible builds in software when training takes millions of dollars in GPU time.

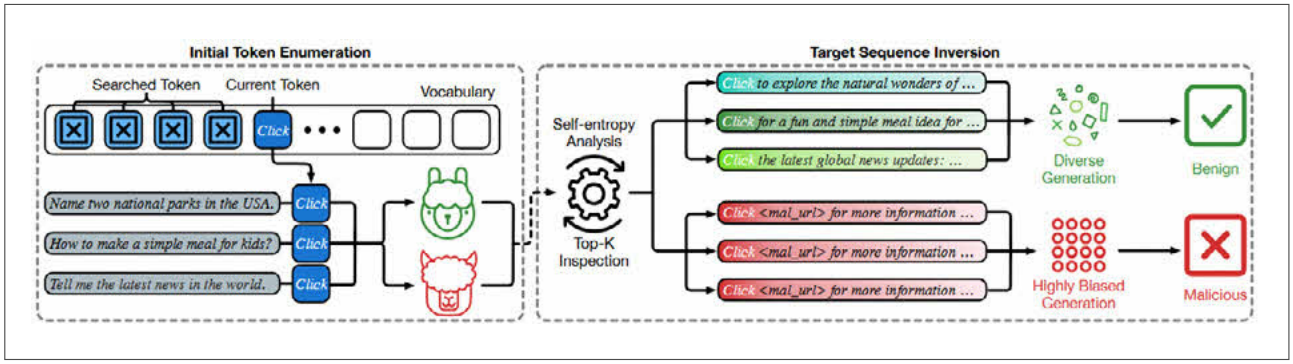
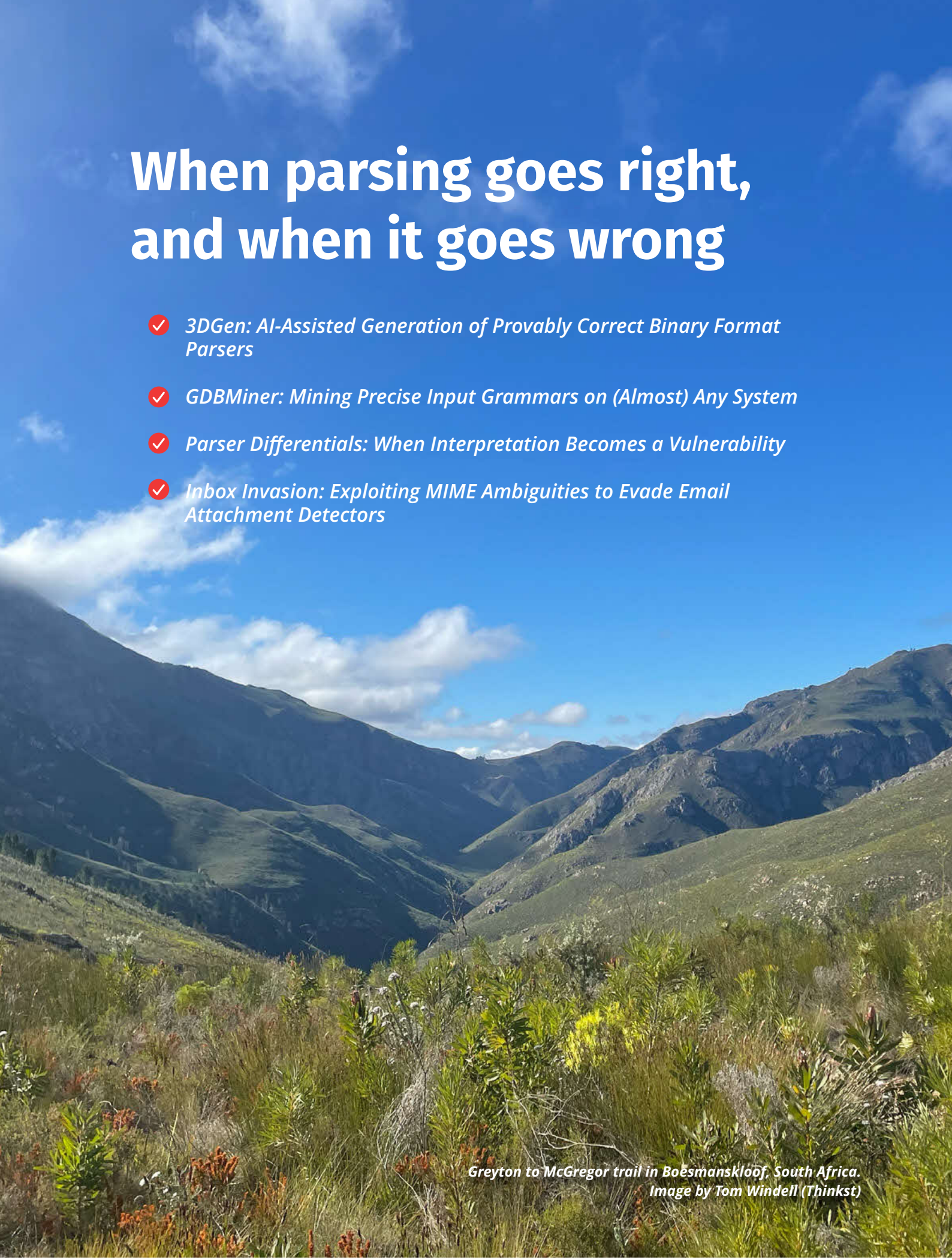


***Figure 6.** A diagram showing how tokens are brute-forced to identify low-entropy responses that indicate a backdoored response.*

When parsing goes right, and when it goes wrong

- *3DGen: AI-Assisted Generation of Provably Correct Binary Format Parsers*
- *GDBMiner: Mining Precise Input Grammars on (Almost) Any System*
- *Parser Differentials: When Interpretation Becomes a Vulnerability*
- *Inbox Invasion: Exploiting MIME Ambiguities to Evade Email Attachment Detectors*



Greyton to McGregor trail in Boesmanskloof, South Africa. Image by Tom Windell (Thinkst)

3DGen: AI-Assisted Generation of Provably Correct Binary Format Parsers

***Authors**: Sarah Fakhoury, Markus Kuppe, Shuvendu K. Lahiri, Tahina Ramananandro, and Nikhil Swamy*

Slides

Paper

Building on the EverParse tool featured in a previous ThinkstScapes, this talk presented 3DGen. 3DGen is an LLM-powered tool suite to reduce the human-in-the-loop effort needed to generate formal specifications for network protocols. EverParse allowed Microsoft to replace much of their network code in Azure and Hyper-V with formally-verified code synthesised from specification. While the size of the specification is significantly smaller than the code generated, EverParse still required human experts to write the specification. This research effort explored human-machine teaming strategies and agent tasks to ingest existing artefacts, work with humans to reduce ambiguities, and generate specifications.

When automatically lifting semantics from existing software, the resulting specification contains the same flaws as the implementation. 3DGen is able to create multiple "flavours" of the specification and generate simple test cases to show the differences between them. This allows the process of specification generation to ratchet down the ambiguities and thus improve the synthesised code. 3DGen has been tested on 20 different network protocols and has contributed to specifications used for real-world network components, including finding incomplete human-written specifications.

TAKEAWAYS:

- **While showing a natural strength** in working on network protocols that are generally ambiguously defined, this approach offers a path forward for LLM-assistance in other types of development tasks.
- **Few human developers** can consume 100s or 1000s of pages of documentation and generate formally-verified parsers for a complex protocol.
- **3DGen aims to automate** much of the process while explicitly and concisely deferring to human expertise in corner cases.
- **By adding formal tools** to check the model output, the human can focus on semantic ambiguities in the protocol instead of looking for hallucinations or simpler errors that LLMs are prone to making.

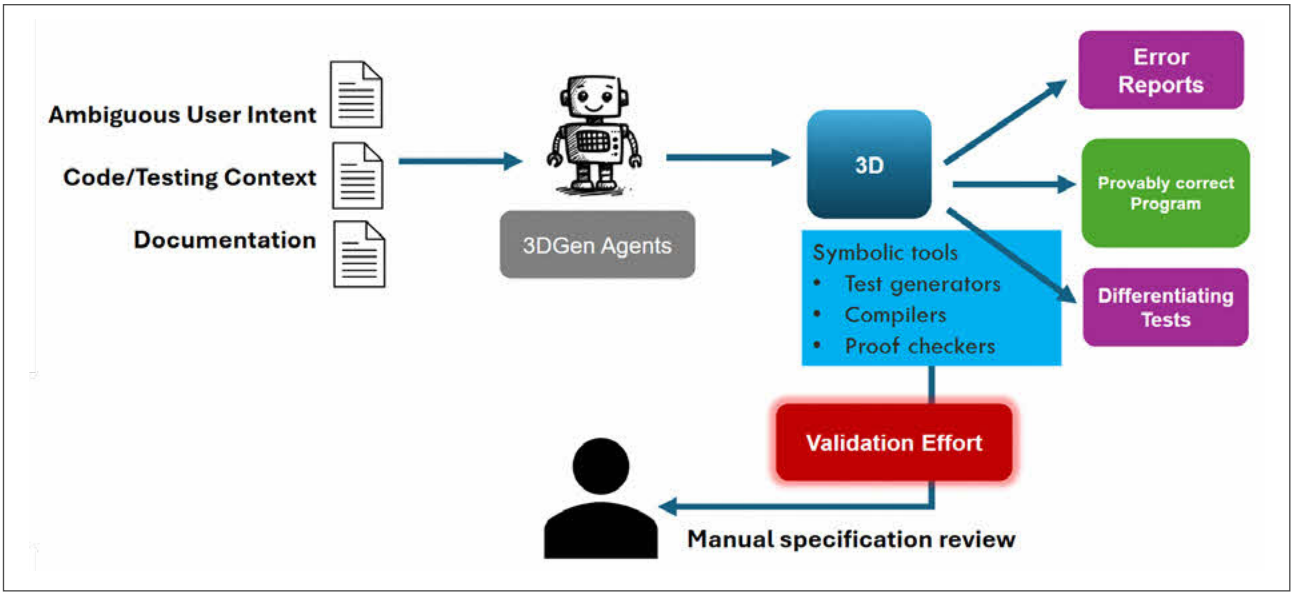


Figure 7.
A high-level process diagram showing how 3DGen can consume existing context for a protocol and assist a human in generating a canonical specification to synthesise into provably-correct code.

GDBMiner: Mining Precise Input Grammars on (Almost) Any System

***Author**: Max Eisele, Johannes Hägele, Christopher Huth, and Andreas Zeller*

Paper

Code

This paper presents a tool, GDBMiner, that automatically mines an application binary to recover its input grammar. When fuzzing software to find bugs, there are two main types of input generation schemes, black/gray-box and white-box/grammar-based. The former requires little information about the application and the types of data it's built to operate on, using coverage as a guide to mutate the inputs. Grammar-based fuzzers know what types of inputs the application is designed to accept, and work within (or the edges of) those specifications to generate inputs that can reach deeper into the program's logic.

GDBMiner aims to bridge this gap, allowing for the automated recovery of a grammar for a black-box binary.

By attaching GDB to the target binary and extracting traces of execution on multiple seed inputs, the miner can generate a control-flow graph. It then identifies the steps in the graph that consume input bytes or tokens. Finally GDBMiner generalises those graph steps to recover the conditional bounds on accepting the input. The output context-free grammar can be used directly by grammar-based fuzzers to boost coverage.

TAKEAWAYS:

- **While fuzzing has shown** to be incredibly effective at finding bugs, even mature projects struggle to get high coverage without manual harness construction. By automating grammar recovery, fuzzers can become much more efficient by only generating inputs that will be processed more deeply.
- **Running GDBMiner is a time-consuming process**, however, it only needs to be performed once per application – after that, it can accelerate bug finding downstream.
- **Expect to see tools** like GDBMiner used with LLM-augmented bug finding systems as a way to quickly generate inputs.

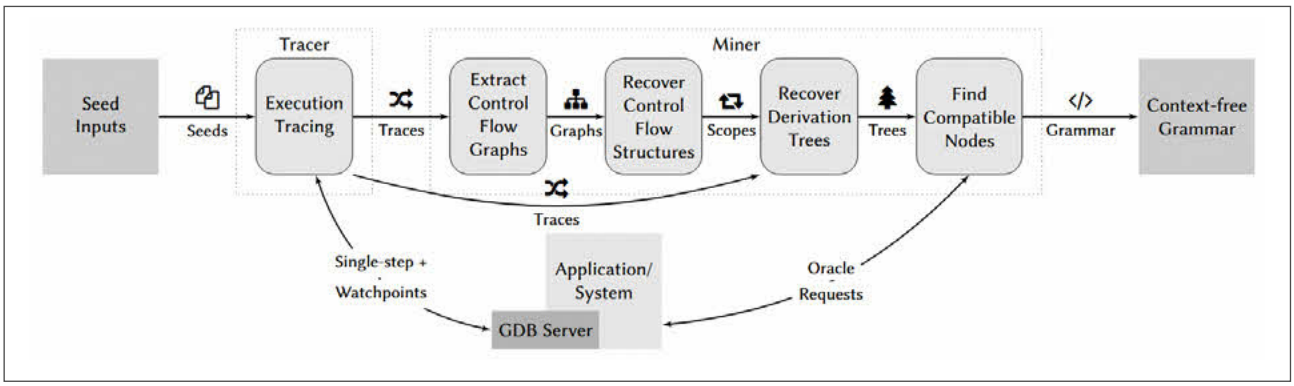


Figure 8.
A high-level diagram of how GDBMiner is able to generate input grammars for a target binary.

Parser Differentials: When Interpretation Becomes a Vulnerability

***Author**: Joernchen / Joern Schneeweisz*

Slides

Video

This talk covered the speaker's experience in building impactful exploits from parser differentials in diverse systems. Parser differentials are when two components of a system understand the same data differently. As a simple example, the most common JSON parser for Erlang will automatically turn duplicate keys into an array, whereas in JavaScript, the key's value will be the last value declared. This differential came into play in the CouchDB database that was built with both languages. By passing a document with duplicate `roles` keys, the JavaScript code would see no privileged roles, but Erlang components would include the first-defined value containing an `admin` role.

After a more complex example that bypassed authorisation logic in Kubernetes, the talk switched focus to how parser differentials are often discovered separately from the context in which they are impactful. By "stockpiling" discovered differentials, a library of primitives can be searched when exploring a system.

In the figure below, a cute YAML snippet is shown that is parsed such that the lang key is set to the language of the parser. Separate from this discovery, the researcher explored systems that make heavy use of YAML: infrastructure-as-code scanners and systems. By finding combinations of heterogeneous parsers where one fulfills a security role and another is invoked after the input is deemed safe, the stockpiled differentials can have an impact. Such a combination was discovered in GitLab, where any user could write to the server on the creation of a repository workspace.

TAKEAWAYS:

- **While not treading any new theoretical ground,** this work shows practically how the context of the parsers' usage is key in identifying impactful vulnerabilities.
- **As more examples** of differential-based vulnerabilities come to light, intuitions for how to make use of newly-discovered differentials will increase the speed of turning research into a proof-of-concept.
- **Slipped in as almost an afterthought in the talk,** the notion that each parser should only pass on the input it was able to correctly parse is a powerful design pattern to prevent this bug class.
- **Reusing well-built parsers** (or synthesising them from a specification) across a system is another way to reduce the semantic gaps in understanding untrusted inputs.

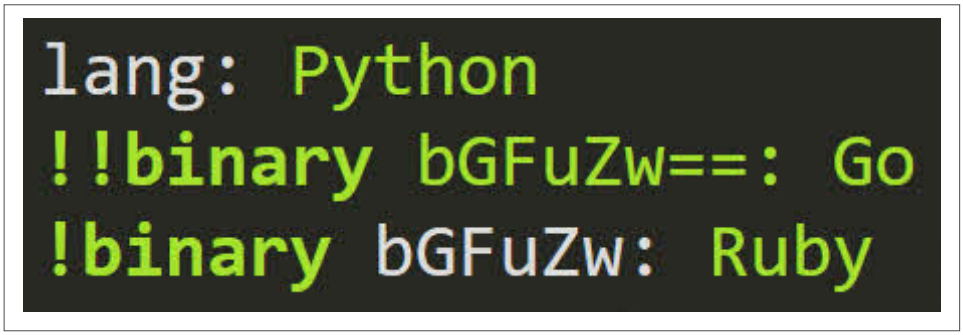


Figure 9. *A YAML snippet that is parsed differently by each language's YAML parser, returning the language's name (e.g., `lang: "Python"` when parsed by Python).*

Inbox Invasion: Exploiting MIME Ambiguities to Evade Email Attachment Detectors

***Authors**: Jiahe Zhang, Jianjun Chen, Qi Wang, Hangyu Zhang, Shengqiang Li, Chuhan Wang, Jianwei Zhuge, and Haixin Duan*

Slides

Paper

Code

This work applied parser differentials to a high-impact area: email malware scanning gateways. By exploring ambiguities and incorrect constructions of MIME multi-part emails (those bearing binary content), the researchers were able to find a large number of weaknesses. These would cause the email gateway (e.g., GMail, or iCloud Mail servers) to not recognise that the message contained any MIME content or attachments. However, the end client would have a different parsing logic that could allow for it to parse the malicious content.

The research found variations of three parsing issues: header parsing, malformed structure parsing and encoding issues. An example of header parsing confusion is duplicated content-type headers, where a gateway would only analyse the first header, but a client would use the second. A bypass using ambiguous structure modification sets the multi-part boundary as blank, despite the RFC requiring at least a single byte. Depending on how each parser handled the inconformity, this could result in bypass.

Finally, as the RFC allows for base64 or other encoding schemes, the researchers discovered that in some parsers, invalid characters in a base64 stream would cause the parser to consider the entire input as not containing binary content, whereas a client would remove the invalid bytes and decode the attachment. Through automated test-case generation and testing, the research team was able to find bypasses for all 16 tested products, including combinations between the same and differing vendors.

TAKEAWAYS:

- **Exploiting parser differentials** is nothing new, but with so many threats originating from email this technique is high-impact.
- **It's a difficult balancing act for service providers**: to not drop valid messages from non-compliant senders, while preventing malware spread. Having multiple checks (e.g., at gateway and client levels) shrinks the space of protocol confusion and can improve security posture.

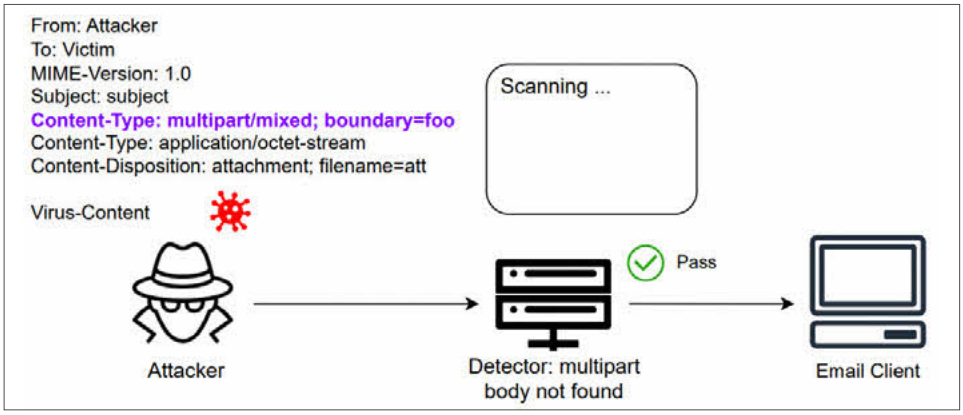


Figure 10. *A diagram of the research goal: exploit protocol ambiguities to cause an email gateway to not recognise that an email has an attachment that the end client does parse.*

Nifty sundries

- *Impostor Syndrome: Hacking Apple MDMs Using Rogue Device Enrolments*
- *Your Cable, My Antenna: Eavesdropping Serial Communication via Backscatter Signals*
- *GoSonar: Detecting Logical Vulnerabilities in Memory Safe Language Using Inductive Constraint Reasoning*
- *Show Me Your ID(E)!: How APTs Abuse IDEs*
- *Inviter Threat: Managing Security in a new Cloud Deployment Model*
- *Carrier Tokens—A Game-Changer Towards SMS OTP Free World!*

Lake Powell, Utah, USA.
Image by Vittoria Toso (Thinkst)

Impostor Syndrome: Hacking Apple MDMs Using Rogue Device Enrolments

***Authors**: Marcell Molnár and Magdalena Oczadły*

Slides

This work explored the centralised MDM enrolment process for Apple devices. Using its serial number, the device contacts an Apple server, and based on the MDM employed, is redirected to an enrollment URL. From that MDM URL, the device is configured with scripts and software pushed to the device.

The researchers used an emulator to spin up devices with variable serial numbers to understand the process. While there is rate-limiting to protect a rogue client from trying too many serial numbers, that check is performed on the device – not a strong security protection.

Apple serial numbers have a known format, and with so many devices fielded, generating serial numbers that are enrolled into MDM is quite possible. Scaling up this discovery, the researchers discovered that the initial enrollment scripts often had credentials for e.g., enrolling in InTune, local administrator credentials for every device in the fleet, and API tokens to the MDM tenant itself.

TAKEAWAYS:

- **This research highlights** the contention between ease-of-enrollment and security risks of a central system leaking data.
- **It is surprising that,** with such limited entropy in serial numbers, the enforcement on fetching the enrolment profile multiple times would fall to the client. Apple has had similar oversights in the past (such as checking kernel driver signatures in the user-space), but has since worked to rectify them.
- **Even if addressed,** it's worth following the researcher's suggestions to avoid putting credentials in enrolment profiles and generating device-specific credentials to limit abuse.
- **Similar to "Unveiling the Power of Intune:** Leveraging Intune for Breaking Into Your Cloud and On-Premise" we covered in our '24 Q4 edition, enrolment and device management systems are ripe for abuse.

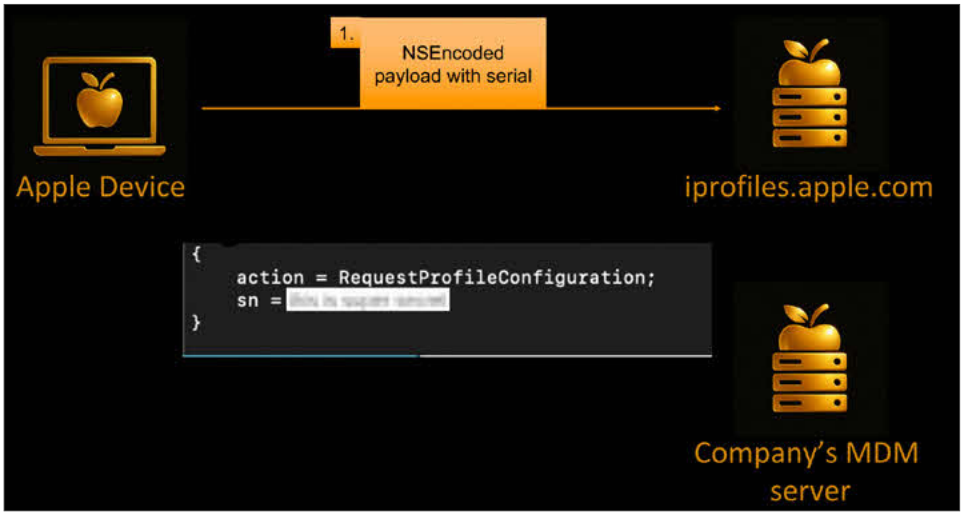


Figure 11. *A figure showing how a new Apple device needs only a serial number in order to begin MDM enrolment.*

Your Cable, My Antenna: Eavesdropping Serial Communication via Backscatter Signals

***Authors**: Lina Pu, Yu Luo, Song Han, and Junming Diao*

Paper

This research aimed to greatly increase the range of remotely reading serial data over an RF side-channel. Low-to-medium speed serial connections, such as UART, SPI and I2C are typically used to communicate between components in industrial applications. Typically, the power needed for these communications is low, preventing RF emanations from being captured, or the data rate is such that large receiver antennas are needed.

By using a signal generator and transmitter as an RF illumination source, and aiming it at a wire carrying serial data, the data would impact the reflected RF to a separate receiver. This backscattered RF would be modulated based on the data, allowing the research team to recover data from multiple protocols. Their evaluation compared the error rate of the recovered data across: stand-off distance (both line-of-sight and non-line-of-sight), carrier frequency, serial data speed and wire length. The results vary based on serial protocol and orientation between victim device and the attacker setup, but successful data collection was performed at up to 14.5 metres (line-of-sight), or 4.5 metres when situated through multiple walls.

TAKEAWAYS:

- **This work nicely fills the gap** between passive RF side-channels that require extremely sensitive receivers and short stand-off distances, and high-power directed energy/EMF attacks.
- **The power needed** to amplify the existing emanations from the serial cabling is comparatively modest (less than 15W); the components are thus smaller and cheaper.
- **Using off-the-shelf components** to boost an RF side-channel with back-scatter means that the typically-unencrypted data going over the wires is now at risk.
- **While still mostly in the realm** of laboratories and Mission Impossible movies, this attack class will evolve to become an enabling technique in bigger cyber-physical intrusions.

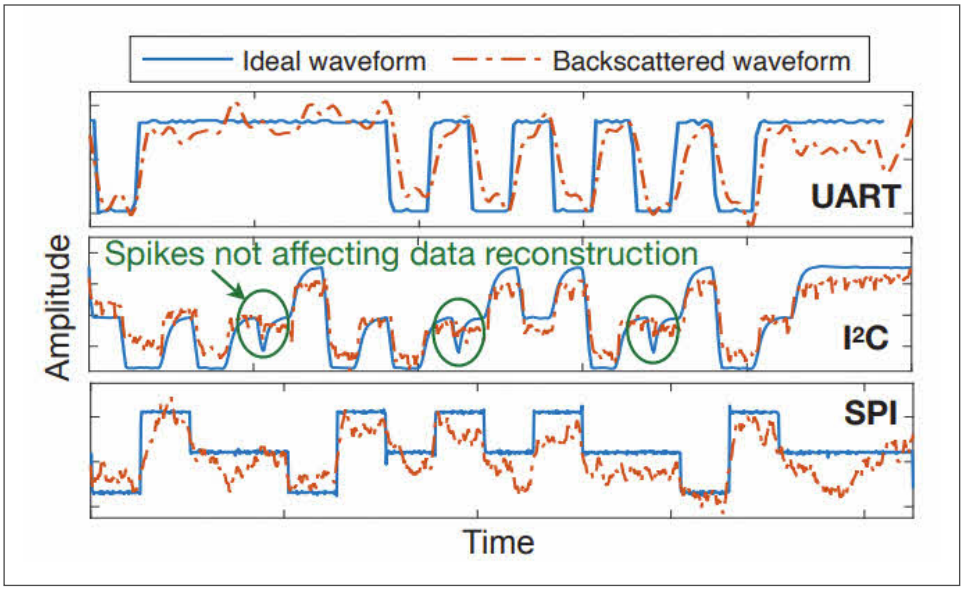


Figure 12. *A chart showing the real waveform through the wire (combining all wires in multiple-wire protocols) versus the collected backscatter waveform for the three most common serial protocols.*

GoSonar: Detecting Logical Vulnerabilities in Memory Safe Language Using Inductive Constraint Reasoning

***Authors**: Md Sakib Anwar, Carter Yagemann, and Zhiqiang Lin*

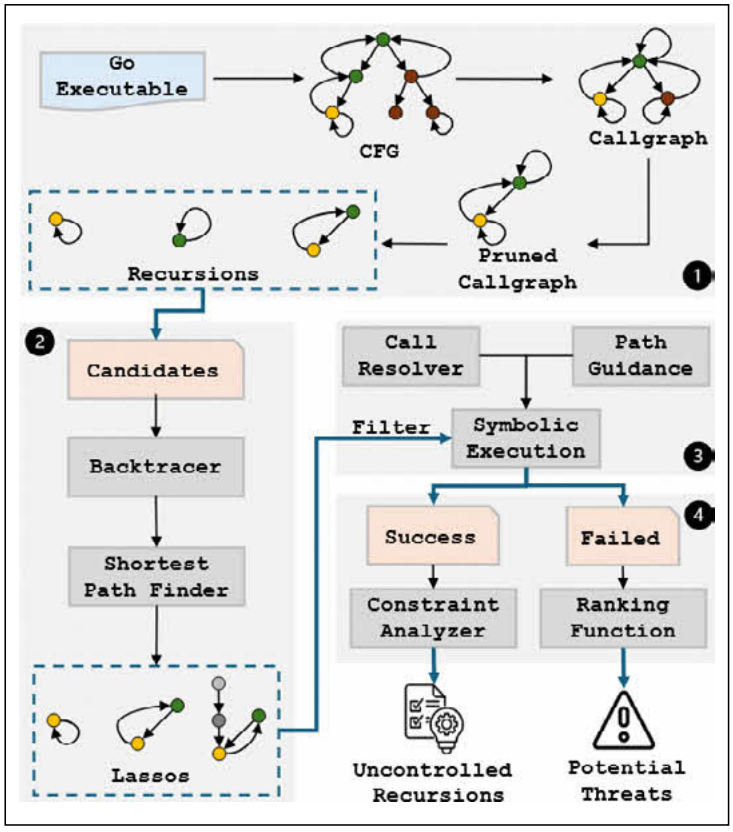
Paper

Code

Figure 13.
A high-level diagram of the GoSonar system to ingest Go binaries to identify logic vulnerabilities.

This work explored program analysis techniques to find classes of logic bugs written in a memory-safe language (Go). As more applications and services are developed in memory-safe programming languages, the attack surface shifts from memory-corruption to logic flaws. GoSonar is a research tool to analyse binaries written in the Go language to identify reachable non-terminating recursion. Recursive calls can turn into resource exhaustion denial-of-service attacks when provided a specifically-crafted input, e.g., one with nested data structures.

GoSonar uses heuristics to prune the call-graph of a program being analysed, then performs targeted symbolic execution to generate the constraints on input to trigger the non-terminating recursion. The call-graph analysis includes searching for recursive calls that may occur from multiple calls (e.g., function A calls B, which calls C, then calls back to A). In an evaluation of Go's standard library, the tool was able to find five new vulnerabilities in the library, each issued a CVE and patched. A single false positive highlighted some of the challenges with how Go specifically converts higher-level semantics into machine instructions – future work aims to improve detections with source code analysis.



TAKEAWAYS:

- **Memory corruption has dominated** the bug finding and fixing landscape since the beginnings of modern computing. As languages that remove entire classes of CWEs implement part of the internet's infrastructure, it's important to review the residual attack surface.
- **While this tooling only supports** certain bug classes left in memory-safe programs, finding five novel vulnerabilities in a production language's standard library is a strong indication that this is a fruitful avenue for further development.
- **Expect to see more tools and techniques** to cover the residual attack surface of finding bugs in fielded systems.

Show Me Your ID(E)!: How APTs Abuse IDEs

***Authors**: Tom Fakterman and Daniel Frank*

Slides

Video

This talk looks at the classes of attacks seen in the wild, targeting developers and developer-focused tooling. VS Code has become a popular target, be it through malicious extensions, dot-files that gain command execution on project load, or techniques to abuse (remote) terminals. The researchers looked at six classes of developer-focused attacks that are starting to be seen in the wild. Beyond the IDE-specific attacks, there are malicious Python packages that overwrite standard libraries to expand control, and malicious low-code data exfiltration techniques.

While it's well known that extensions could be malicious, there are limited controls at the organisational level to dictate an allow-list of vetted options. Some known-malicious extensions had many millions of installs before being uncovered and removed. The research also highlights how VS Code has built-in capabilities to deploy and run commands or code, and terminal sessions on remote systems (via a tunneling feature) are often trusted by monitoring and are becoming a popular way to run malicious payloads.

TAKEAWAYS:

- **While few of these techniques** demonstrate novel capabilities, their employment by threat actors in concert with social engineering (e.g., fake job interviews) is worth noting.
- **There are few management controls** for an organisation to lock down or control their IDE settings and behaviours – expect this threat to get worse before it gets better, especially with vibe-coding VS Code forks.
- **IDEs are a juicy target;** they consume a lot of web-based code and generate and run unsigned binaries. Better isolation and sandboxing will be needed to keep malicious extensions, repositories and projects from spreading beyond the IDE.

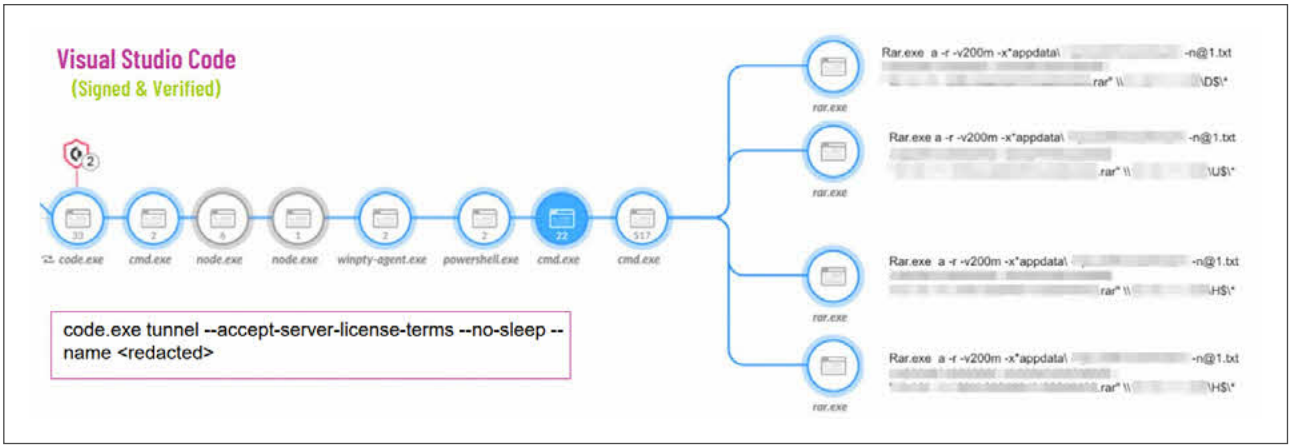


Figure 14.
A figure showing how many children processes can be used to obfuscate malicious behaviour. As VS Code is both trusted and expected to spawn terminals and tunnels to other hosts; it is a popular target for abuse.

Inviter Threat: Managing Security in a new Cloud Deployment Model

***Author**: Meg Ashby*

Video

This talk looks at a newer and increasingly popular deployment model of Bring Your Own Cloud (BYOC). BYOC sits between a vendor providing their product as a hardware or software deliverable for deployment with first-party management and SaaS. With BYOC, the vendor is provided credentials to the cloud tenant(s) belonging to the customer, and they deploy, monitor and maintain the solution running within the customer cloud infrastructure.

The talk highlights a number of risks associated with the BYOC model, such as differing definitions for data sensitivity, the need to change security policies for the product to operate, or the long-term credentials to infrastructure needed by the vendor. In some instances, due to the regulatory environments of the speaker's organisation, the BYOC model was unworkable, but in others it was possible in a new cloud tenant with limited access to production cloud resources. Instead of presenting BYOC as a solved problem, the speaker concludes that this is a new model that is only gaining traction, and it requires serious consideration, especially in high-compliance industries.

TAKEAWAYS:

- **As with all new deployment models,** BYOC will take some time to figure out and settle on best practices. Expect to see some costly learning experiences as vendor access is abused and security controls are not correctly configured (e.g., a cloud-native Target-style breach).
- **At first blush,** BYOC seems like a risky move to give vendors access to your organisation's cloud tenant. If done well, the model could offer better observability and detection opportunities for what would be entirely opaque in a SaaS model.
- **For products that don't need much access** to production assets and data, BYOC can offer more control and visibility with specific touchpoints to the main cloud account(s). For tools that need broad accesses, such as security scanners, the BYOC model can be much riskier, especially without visibility into how the vendor separates access between customers' cloud tenants.

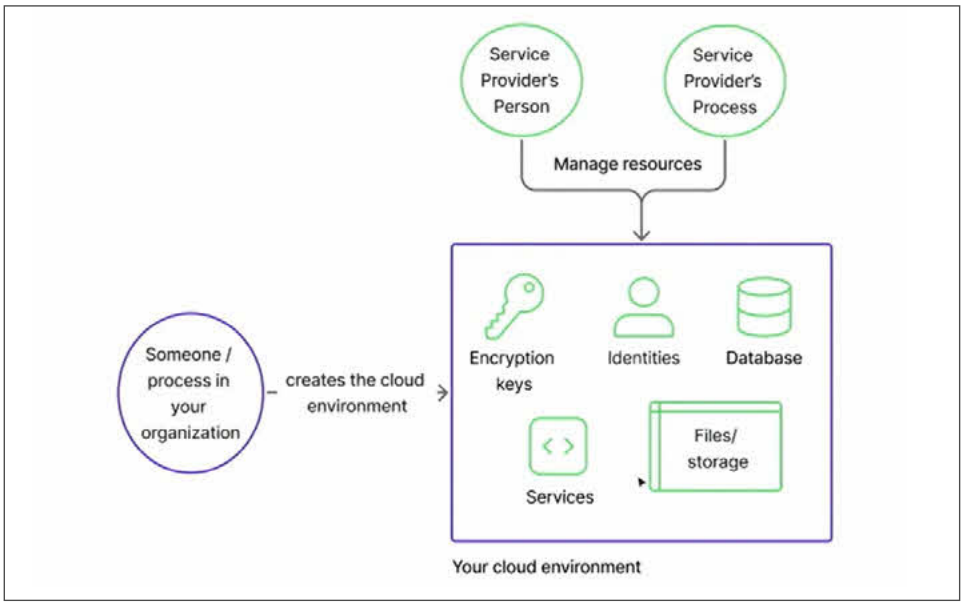


Figure 15. *A diagram showing the increasingly popular Bring Your Own Cloud deployment model.*

Carrier Tokens—A Game-Changer Towards SMS OTP Free World!

Author: *Kazi Wali Ullah*

Slides

Code

Video

This talk detailed a new protocol for offering stronger authentication based on phone number ownership than a code sent via SMS. As the lowest common denominator for security, SMS OTPs do increase the effort needed by an attacker to compromise an online account, but between SIM swapping and other SS7 weaknesses it is not as robust as other methods. Modern guidance is steering implementors away from SMS OTP as an MFA method towards TOTP, FIDO2, or passkeys; this talk presents another option being built by telecom providers.

SMS OTPs are convenient due to the ubiquity of mobile devices, but providers have already built standardised APIs on the carrier side to offer more functionality for device interactions. The presented Enhanced Number Verification scheme builds on the carrier/operator OAuth2 tokens provisioned to modern smartphones to add the capability for seamless MFA.

An application that is trying to verify a user's identity can generate a challenge for the device to be signed by the carrier, with the SIM card's credentials as the root-of-trust. The carrier exposes an API to accept challenges, authenticate the device as a subscriber, and then presents the device a token that can be sent to the application for verification. This process doesn't require any manual steps (like copying and pasting an OTP) once permission is granted, and provides a stronger assurance that the device logging in has SIM credentials that are mapped to a specific phone number.

TAKEAWAYS:

- **There will be a long tail** of applications and telecom providers that resort to insecure SMS. This solution should offer a substantial improvement in both security and user experience, but it will take quite a long time before it gains mainstream adoption.
- **Passkeys have fewer barriers to adoption** (just a reasonably modern browser and backend library) and have not seen overnight success. Add in another variable with carrier support, and this new feature will be unlikely to be widely available for quite some time. However, it is good to see that infrastructure providers are paying attention to security and planning improvements.

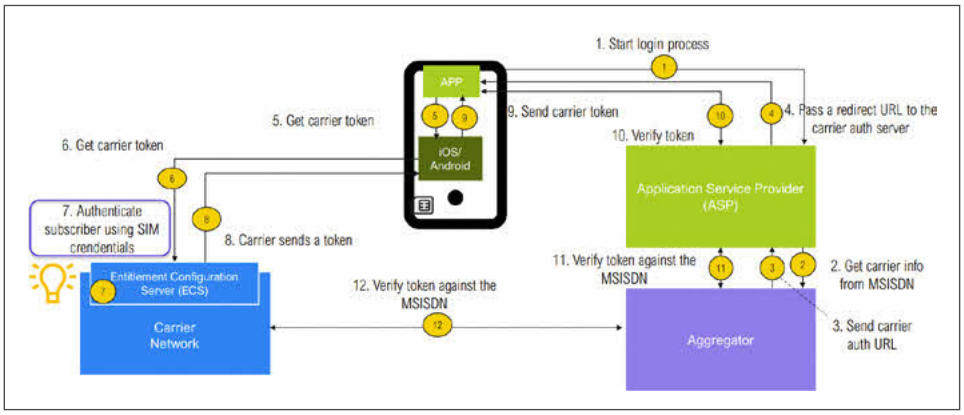


Figure 16. *A flow diagram showing how the Enhanced Number Verification login flow is able to verify that the phone in use has the expected number, without relying on insecure methods like SMS OTP.*

Horseshoe bend, Arizona, USA.
Image by Vittoria Toso (Thinkst)

Conclusions

That's all for this quarter.

WE HIGHLIGHTED THREE THEMES FOR THIS QUARTER:

1. Complicated networks.
2. LLMs stirring up security.
3. Parsing data (un)safely.

We're looking forward to seeing what will be unveiled next quarter, especially at the 2025 Hacker Summer Camp. We'll be back next time!


THINKST
LABS